Git Hooks Cheat Sheet for Junior Developers

Git hooks are automated scripts that execute before or after core Git commands like commit and push. They help catch syntax errors, block leaked credentials, and maintain clean repositories without manual effort.

What you'll learn:

1. Creating Your First Pre-Commit Hook
2. Blocking Leaked Secrets and Passwords
3. Sharing Hooks with Your Team via Husky
4. Enforcing Ticket Numbers in Commit Messages

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Creating Your First Pre-Commit Hook

Git creates a hidden folder called .git/hooks in every repo with sample scripts. To make a hook active, remove the .sample extension or create a new file named 'pre-commit'. Add your bash commands, like running a test or linter. Finally, make the file executable so Git has permission to run it automatically before each commit.

```
$echo '#!/bin/sh
 npm test' > .git/hooks/pre-commit && chmod +x .git/hooks/pre-commit
```

Git Hooks: Automate Checks Before Every Commit

Saturday, October 17, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Blocking Leaked Secrets and Passwords

Hardcoded secrets are one of the most common causes of cloud security breaches. You can block them locally by running Gitleaks inside your pre-commit hook. The command inspects only your staged changes for known secret patterns. If an API key or private token is spotted, the commit aborts instantly before reaching GitHub.

```
$gitleaks protect --staged --verbose
```

Git Hooks: Automate Checks Before Every Commit

Saturday, October 17, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Sharing Hooks with Your Team via Husky

Files placed inside .git/hooks are ignored by remote repositories and stay on your local machine. Husky solves this problem for JavaScript and Node-based projects by storing hook definitions in a shared .husky directory. When your teammates clone the repo and install dependencies, the hooks are automatically activated for them too.

```
$npx husky init && echo "npm run lint" > .husky/pre-commit
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Enforcing Ticket Numbers in Commit Messages

The commit-msg hook inspects the text file where Git temporarily stores your commit message. You can use a simple bash script with regex to confirm that the message starts with an approved ticket prefix. If a developer types a vague message like 'fixed bug', Git rejects the commit and asks for proper formatting.

```
$#!/bin/sh
 grep -Eq '^(PROJ-[0-9]+|HOTFIX): ' "$1" || { echo 'Invalid commit message format!'; exit 1; }
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Keep pre-commit checks fast (under 5 seconds) to avoid frustrating developers.
+ Always make hook scripts executable on Linux and macOS using chmod +x.
+ Store hook configurations in your repository using Husky or the Python pre-commit framework.
+ Use pre-push instead of pre-commit for heavy end-to-end integration test suites.
+ Reserve the --no-verify bypass flag exclusively for genuine production emergencies.

Watch Out For

! Assuming .git/hooks files sync with GitHub; standard git hooks are strictly local to your machine.
! Writing long, slow hook scripts that prompt developers to permanently disable or bypass them.
! Forgetting to make scripts executable, causing Git to silently ignore the hook without an error.
! Relying entirely on local hooks for security; always keep secondary CI/CD checks running on GitHub.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh